DOCUMENTO TECNICO DI ARCHITETTURA SOFTWARE

Analisi Dettagliata del Modulo di Fisica Z80 physics.asm

Motore di gioco ZX Spectrum – Project *Jet Set Willy*

Oggetto del Documento: *Questo manuale fornisce una trattazione approfondita ed esaustiva dell'intero motore fisico contenuto nel file physics.asm per il gioco Jet Set Willy (ZX Spectrum Z80). Vengono esaminate la decompressione dei layout, le collisioni a livello di pixel con terreno, muri e pericoli, la dinamica di salto e gravita a precisione frazionaria, il calcolo della pendenza delle rampe, l'effetto dei nastri trasportatori e le transizioni tra stanze.*

Tutte le sezioni includono le relative spiegazioni teoriche e i ***blocchi di codice sorgente Z80 Assembly integrali e commentati*** *tratti da physics.asm.*

1. Architettura Generale e Strutture Dati

Il modulo physics.asm gestisce tutta la dinamica di gioco per lo sprite del protagonista (Willy), interagendo con la griglia di sfondo di ciascuna stanza (256x128 pixel divisa in 32 colonne x 16 righe di celle 8x8) e con le entita dinamiche come nemici e nastri trasportatori.

- **Coordinate e Dimensioni:** L'area giocabile e di 256 pixel in larghezza (X in [0, 255]) e 128 pixel in altezza (Y in [0, 127]). Willy ha una dimensione di 16 x 8 pixel (altezza 16, larghezza 8).
- **Buffer di Layout Decompresso (unpacked_layout):** I dati originali della stanza occupano 128 byte in ROM/RAM dove ogni cella e codificata con 2 bit (4 celle per byte). All'ingresso di una stanza, la routine UnpackRoomLayout espande tali dati in 512 byte (1 byte per cella). Questo garantisce un accesso casuale O(1) immediato senza dover decodificare bit per bit ad ogni frame.
- **Valori delle Celle (Tipi di Blocco):**
  - **0 (Empty):** Cella vuota traversabile.
  - **1 (Floor):** Terreno/Pavimento solido.
  - **2 (Wall):** Muro solido (blocca movimento orizzontale e salita).
  - **3 (Nasty):** Elemento letale/pericoloso (causa la morte immediata al contatto).
- **Flag di Terreno (on_ground):** Indica se Willy si trova su una superficie solida (valore 1) oppure se e in volo/caduta (valore 0).
- **Velocita e Gravita (velocity_y):** Valore signed a 8 bit che rappresenta la velocita verticale (positiva verso l'alto durante il salto, negativa verso il basso in caduta).

2. Decompressione Layout e Lookup Celle O(1)

Per ottimizzare le prestazioni su Z80 (3.5 MHz), il layout compresso a 2-bit per cella viene espanso in un array continuo di 512 byte nella RAM tramite UnpackRoomLayout. Successivamente, la funzione GetCellFromUnpacked calcola l'indirizzo Row * 32 + Col in pochissimi cicli macchina.

```
UnpackRoomLayout:
    push hl : push de : push bc
    ld   hl, (room_ptr)     ; HL = inizio layout stanza (128 byte)
    ld   de, unpacked_layout ; DE = buffer RAM (512 byte)
    ld   b, 128             ; 128 byte
.byte_loop:
    ld   a, (hl)            ; Leggi byte compresso
    inc  hl
    ld   c, a               ; C = salvataggio

    ; Cella 0: bit 7-6
    rlca : rlca : and 3 : ld (de), a : inc de
    ; Cella 1: bit 5-4
    ld a, c : rlca : rlca : rlca : rlca : and 3 : ld (de), a : inc de
    ; Cella 2: bit 3-2
    ld a, c : rrca : rrca : and 3 : ld (de), a : inc de
    ; Cella 3: bit 1-0
    ld a, c : and 3 : ld (de), a : inc de

    djnz .byte_loop
    pop bc : pop de : pop hl
    ret

GetCellFromUnpacked:
    push hl : push de
    ld   l, a
    ld   h, 0
    add  hl, hl : add hl, hl : add hl, hl : add hl, hl ; HL = Row * 32
    ld   e, c : ld d, 0
    add  hl, de             ; HL = Row * 32 + Col
    ld   de, unpacked_layout
    add  hl, de
    ld   a, (hl)            ; A = cell type (0..3)
    pop  de : pop hl
    ret
```

3. Rilevamento di Solidi, Muri e Pericoli (Nasty Cells)

Il motore fisico distingue tre tipi di verifiche d'impatto:

1. **Phys_CheckSolid:** Determina se un punto (X,Y) e solido (Floor = 1 o Wall = 2), considerando anche le celle occupate da nastri trasportatori e rampe. Restituisce il flag Carry impostato se solido.

2. **CheckWillyWallCollision:** Effettua un controllo orizzontale su 3 punti (testa Y, centro Y+7, piedi Y+15) per verificare se Willy sta urtando frontalmente contro un muro (Wall = 2), impedendone il movimento.

3. **Phys_CheckNastyCollision:** Verifica la morte di Willy tramite 6 punti di controllo (Top-Left, Top-Right, Mid-Left, Mid-Right, Bottom-Left, Bottom-Right) attorno alla bounding box di 16 x 8 pixel.

```
Phys_CheckSolid:
    push hl : push de : push bc
    ld   a, b
    bit  7, a               ; Se Y < 0, non e solido
    jr   nz, .not_solid
    cp   128                ; Clamp Y se >= 128
    jr   c, .y_ok
    ld   b, 127
.y_ok:
    call CheckConveyorOrRampCell
    jr   c, .solid

    ld   a, b : srl a : srl a : srl a   ; r = Y / 8
    ld   e, a
    ld   a, c : srl a : srl a : srl a   ; c = X / 8
    ld   c, a : ld a, e
    call GetCellFromUnpacked

    cp   1                  ; Floor?
    jr   z, .solid
    cp   2                  ; Wall?
    jr   z, .solid
.not_solid:
    pop bc : pop de : pop hl : or a : ret
.solid:
    pop bc : pop de : pop hl : scf : ret

Phys_CheckNastyCollision:
    push bc
    ld   bc, (PlayerPos)    ; B = Y, C = X
    call CheckNastyCell : jr c, .collision              ; Top-Left (X, Y)
    ld   a, c : add a, 7 : ld c, a : call CheckNastyCell ; Top-Right (X+7, Y)
    jr   c, .collision
    ld   bc, (PlayerPos) : ld a, b : add a, 7 : ld b, a
    call CheckNastyCell : jr c, .collision              ; Mid-Left (X, Y+7)
    ld   a, c : add a, 7 : ld c, a : call CheckNastyCell ; Mid-Right (X+7, Y+7)
    jr   c, .collision
    ld   bc, (PlayerPos) : ld a, b : add a, 15 : ld b, a
    call CheckNastyCell : jr c, .collision              ; Bottom-Left (X, Y+15)
    ld   a, c : add a, 7 : ld c, a : call CheckNastyCell ; Bottom-Right (X+7, Y+15)
    jr   c, .collision
    pop  bc : or a : ret
.collision:
    pop  bc : scf : ret
```

4. Gravita, Salti e Meccaniche di Movimento (Phys_ApplyGravity)

La routine principale del motore fisico e Phys_ApplyGravity. Essa gestisce:

- **Interpolazione della Gravita (Preset):** Il motore supporta 7 preset di gravita configurabili per stanza (0.5, 0.75, 1.0, 0.25, 0.0, 0.875, 0.625). L'accelerazione verso il basso viene applicata a intervalli regolari alternando i frame mediante un contatore jump_frame e operazioni di maschera bit a bit (AND).
- **Meccanica Step-Down:** Quando Willy cammina in discesa su gradini o rampe, il sistema rileva se il piede avanzato si trova nel vuoto ma il livello sottostante (Y+24) e solido. In tal caso applica uno Step-Down immediato di +8 pixel, evitando che Willy fluttui in aria prima di iniziare a cadere.
- **Collisione con Soffitti (Ceiling Collision):** Durante la fase ascendente del salto (velocity_y > 0), il motore controlla se la testa di Willy impatta contro blocchi di tipo Wall. In caso d'impatto, azzera la velocita verticale e clamp la posizione Y al bordo inferiore del muro.
- **Atterraggio sul Terreno:** Durante la caduta (velocity_y <= 0), vengono controllati i due punti sotto i piedi (X, Y+16 e X+7, Y+16). Se viene rilevato solido e la posizione precedente era sopra o a livello della superficie (tolleranza di 3 pixel), Willy viene posizionato a terra (on_ground = 1).
- **Transizioni di Stanza Verticali:** Se Willy supera il bordo superiore (Y < 0) o inferiore (Y >= 113), il motore controlla la mappa delle stanze adiacenti e scatena la routine TriggerRoomChange.

```
Phys_ApplyGravity:
    ld   bc, (PlayerPos)
    ld   a, (on_ground) : or a : jr z, .not_on_ramp_walk

    call GetRampY           ; Aggancio rampa durante la camminata
    jr   nc, .not_on_ramp_walk
    ld   e, a : ld a, b : sub e
    bit  7, a : jr z, .abs_ok : neg
.abs_ok:
    cp   12 : jr nc, .not_on_ramp_walk
    ld   b, e               ; B = ramp_Y
    ld   (PlayerPos), bc

    ld   a, (jump_pressed)  ; Gestisci eventuale salto da rampa
    or   a : jp z, .end
    ; Imposta parametri del salto...
    jp   .in_air

.not_on_ramp_walk:
    ; Controllo Step-Down (discesa da gradini di 8px)
    ; Applicazione gravita frazionaria basata su room_gravity_preset...
    ld   a, (jump_frame) : inc a : ld (jump_frame), a
    ; applica diminuzione di velocity_y secondo preset...

.clamp_fall_speed:
    cp   -MAX_FALL_SPEED    ; Limitatore di caduta (max -12)
    jr   nc, .store_vel
    ld   a, -MAX_FALL_SPEED
.store_vel:
    ld   (velocity_y), a
    ; Aggiorna posizione Y = Y - velocity_y
    ...
```

5. Matematica delle Rampe e Calcolo Trigonometrico Lineare

Le rampe in Jet Set Willy sono superfici inclinate a 45 gradi che collegano quote differenti. L'algoritmo contenuto in GetRampY determina in tempo reale la coordinata Y esatta corrispondente a qualsiasi coordinata X di Willy.

Formule di Calcolo della Rampa (GetRampY):

- **Rampa che sale verso Destra (ramp_dir = 1):**
  X_left = ramp_col * 8
  dX = X_willy - X_left
  Y_ramp = (ramp_row * 8 - 16) - dX

- **Rampa che sale verso Sinistra (ramp_dir = 0):**
  X_start = ramp_col * 8
  dX = X_start - X_willy
  Y_ramp = (ramp_row * 8 - 16) - dX

Protezione Guardia d'Ingresso (ramp_entry_guard):
Quando Willy entra in una stanza in prossimita del bordo inferiore sottostante una rampa, la guardia attiva impedisce che Willy venga 'risucchiato' o agganciato erroneamente alla rampa sopra di lui finche non si allontana dalla zona d'ingresso o si allinea volontariamente.

```
GetRampY:
    ld   a, (ramp_len) : or a : ret z   ; Nessuna rampa? Exit carry clear
    ld   a, (ramp_dir) : or a : jr z, .dir_left

.dir_right:
    ld   a, (ramp_col) : add a,a : add a,a : add a,a : ld e, a ; E = X_left
    ld   a, c : cp e : jp c, .out_of_ramp                      ; PlayerX < X_left?

.on_ramp_slope:
    ld   a, c : sub e : ld d, a                                ; D = dx (X - X_left)
    ld   a, (ramp_row) : add a,a : add a,a : add a,a : sub 16  ; Base Y
    sub  d                                                     ; Y = BaseY - dx
    ld   d, a
    ; Clamp alla piattaforma di cima se necessario...
    jp   .return_ramp_y

.dir_left:
    ; Calcolo speculare per rampe a sinistra...
    ...

.return_ramp_y:
    ; Controllo ramp_entry_guard per evitare agganci errati ai cambi stanza...
    ld   a, (ramp_entry_guard) : or a : jr z, .guard_clear_exit
    ; Se la differenza Y > 12px ed e sotto la rampa -> ignora rampa!
    ...
.guard_clear_exit:
    ld   a, b : pop bc : scf : ret
.out_of_ramp:
    or   a : ret
```

6. Nastri Trasportatori e Collisioni con Nemici

Nastri Trasportatori (Phys_ApplyConveyor):
Se Willy si trova a terra (on_ground = 1) sopra una cella appartenente alla riga e colonna del nastro trasportatore (conveyor_row, conveyor_col), la routine lo trascina automaticamente di 1 pixel per frame nella direzione definita (conveyor_dir). Se il nastro spinge Willy oltre il bordo dello schermo, viene innescato il cambio di stanza orizzontale.

Collisione Bounding Box Nemici (CheckCollision):
Cicla su tutti gli 8 slot dei nemici attivi (ActiveEnemies). Esegue una verifica rettangolare AABB con tolleranza pixel su orizzontale e verticale:

```
CheckCollision:
    ld   ix, ActiveEnemies
    ld   b, 8               ; 8 slot nemici
.loop:
    ld   a, (ix+0) : and $80 ; Nemico attivo?
    jr   z, .next

    ; Controllo orizzontale: PlayerX - EnemyX + 7 in [0, 22]
    ld   a, (PlayerPos) : sub (ix+2) : add a, 7
    cp   23 : jr nc, .next

    ; Controllo verticale: PlayerY - EnemyY + 15 in [0, 30]
    ld   a, (PlayerPos+1) : sub (ix+3) : add a, 15
    cp   31 : jr nc, .next

    ; Collisione trovata!
    pop ix : pop bc : pop de : pop hl : scf : ret
.next:
    ld de, 11 : add ix, de  ; Prossimo nemico (+11 byte)
    djnz .loop
    pop ix : pop bc : pop de : pop hl : or a : ret
```

7. Appendice: Sorgente Completo Integrale physics.asm

Di seguito viene allegato il codice sorgente integrale contenuto nel file physics.asm (1891 righe), per consultazione immediata e verifica di ciascun dettaglio di implementazione Z80 Assembly.

```
;===============================================================================
; physics.asm
;===============================================================================


; ---------------------------------------------------------------------
; Verifica se c'è una cella solida (Floor o Wall) alle coordinate pixel (X, Y)
; Input: C = X, B = Y
; Output: Carry flag impostato se solida, altrimenti resettato
; ---------------------------------------------------------------------
Phys_CheckSolid:
    push hl
    push de
    push bc

    ; Se Y < 0 (sopra lo schermo), non è solida
    ld   a, b
    bit  7, a
    jr   nz, .not_solid

    ; Se Y >= 128, clampiamo a 127 per poter controllare il pavimento (riga 15)
    cp   128
    jr   c, .y_ok
    ld   b, 127
.y_ok:

    call CheckConveyorOrRampCell
    jr   c, .solid

    ; 2. Calcola riga di layout: r = Y / 8
    ld   a, b
    srl  a
    srl  a
    srl  a                  ; A = r (0..15)

    ; 3. Calcola colonna di layout: c = X / 8 (in C)
    ld   e, a               ; E = r
    ld   a, c
    srl  a
    srl  a
    srl  a                  ; A = c (0..31)
    ld   c, a               ; C = c
    ld   a, e               ; A = r

    call GetCellFromUnpacked ; A = cell type

    ; 5. Controlla se è solida: Floor (1) o Wall (2)
    cp   1
    jr   z, .solid
    cp   2
    jr   z, .solid

.not_solid:
    pop  bc
    pop  de
    pop  hl
    or   a                  ; Clear carry
    ret

.solid:
    pop  bc
    pop  de
    pop  hl
    scf                     ; Set carry
    ret


; ---------------------------------------------------------------------
; Applica gravità e gestisce salto
; ---------------------------------------------------------------------
Phys_ApplyGravity:
    IF ENABLE_RAMP_DEBUG_BARS != 0
    xor  a
    ld   (willy_on_ramp), a
    ENDIF
    ld  bc, (PlayerPos)
    ld  a, b              ; B = PlayerY (pixel)

    ; --- CONTROLLO RAMPA PER IL MOVIMENTO ---
    ld   a, (on_ground)
```

```
    or   a
    jr   z, .not_on_ramp_walk ; Se siamo in volo, non fare il snap della camminata sulla rampa

    call GetRampY           ; C = PlayerX, A = ramp_Y (se carry set)
    jr   nc, .not_on_ramp_walk

    ; Siamo nel range della rampa!
    ; Controlliamo se siamo abbastanza vicini verticalmente
    ld   e, a               ; E = ramp_Y
    ld   a, b               ; A = PlayerY
    sub  e                  ; A = PlayerY - ramp_Y
    bit  7, a
    jr   z, .abs_ok
    neg
.abs_ok:
    cp   12                 ; Soglia di vicinanza di 11 pixel
    jr   nc, .not_on_ramp_walk ; Se troppo lontano, non agganciare la rampa

    ; Aggancia sulla rampa!
    ld   b, e               ; B = ramp_Y
    bit  7, b
    jp   nz, .ramp_top_exit ; Se B è negativo (bit 7 = 1), Willy è in cima alla rampa al bordo superiore!
    ld   (PlayerPos), bc    ; Salva la posizione Y allineata alla rampa
    IF ENABLE_RAMP_DEBUG_BARS != 0
    ld   a, 1
    ld   (willy_on_ramp), a
    ENDIF

    ; Gestisci il salto mentre siamo sulla rampa
    ld   a, (jump_pressed)
    or   a
    jp   z, .end            ; Se non salta, abbiamo finito per questo frame

    xor  a
    ld   (Audio_jump_sound_note), a
    ld   (Audio_jump_sound_timer), a

    ld   a, (room_jump_strength)
    ld   (velocity_y), a
    xor  a
    ld   (on_ground), a
    ld   (jump_pressed), a
    ld   (jump_frame), a
    ld   a, b              ; B = PlayerY (ramp_Y)
    ld   (jump_start_y), a
    ld   a, (current_input_dir)
    ld   (jump_dir), a
    jp   .in_air

.not_on_ramp_walk:
    ld  bc, (PlayerPos)
    ld  a, b             ; B = PlayerY (pixel)

    ; Controlla se siamo a mezz'aria (on_ground == 0)
    ld  a, (on_ground)
    or  a
    jp  z, .in_air        ; Se siamo in volo, continua a volare

    ; --- CONTROLLO STEP DOWN DI 1 TILE (8 pixel) ---
    ; Se stiamo scendendo da una rampa o gradino di 1 tile,
    ; facciamo scendere Willy immediatamente per evitare che fluttui in aria.
    ld   a, (direction)
    cp   1
    jr   z, .stepping_left

.stepping_right:
    ; Muovendo a destra: piede sinistro (dietro) a Y + 16 deve essere solido
    ld   a, b
    add  a, 16
    push bc
    ld   b, a              ; B = Y + 16
    call Phys_CheckSolid    ; C = X
    pop  bc
    jr   nc, .no_step_down

    ; Piede destro (davanti) a Y + 16 deve essere vuoto
    ld   a, b
    add  a, 16
    push bc
    ld   b, a              ; B = Y + 16
```

```
    ld   a, c
    add  a, 7
    ld   c, a              ; C = X + 7
    call Phys_CheckSolid
    pop  bc
    jr   c, .no_step_down

    ; Piede destro (davanti) a Y + 24 deve essere solido
    ld   a, b
    add  a, 24
    cp   128
    jr   nc, .no_step_down
    push bc
    ld   b, a              ; B = Y + 24
    ld   a, c
    add  a, 7
    ld   c, a              ; C = X + 7
    call Phys_CheckSolid
    pop  bc
    jr   nc, .no_step_down

    jr   .do_step_down

.stepping_left:
    ; Muovendo a sinistra: piede destro (dietro) a Y + 16 deve essere solido
    ld   a, b
    add  a, 16
    push bc
    ld   b, a              ; B = Y + 16
    ld   a, c
    add  a, 7
    ld   c, a              ; C = X + 7
    call Phys_CheckSolid
    pop  bc
    jr   nc, .no_step_down

    ; Piede sinistro (davanti) a Y + 16 deve essere vuoto
    ld   a, b
    add  a, 16
    push bc
    ld   b, a              ; B = Y + 16
    call Phys_CheckSolid    ; C = X
    pop  bc
    jr   c, .no_step_down

    ; Piede sinistro (davanti) a Y + 24 deve essere solido
    ld   a, b
    add  a, 24
    cp   128
    jr   nc, .no_step_down
    push bc
    ld   b, a              ; B = Y + 24
    call Phys_CheckSolid    ; C = X
    pop  bc
    jr   nc, .no_step_down

.do_step_down:
    ; Controlla se il nuovo Y (B + 8) causerebbe collisioni con muri a sinistra o a destra.
    ; Se sì, annulla lo step down per evitare che Willy sprofondi/entri nei blocchi Wall.
    ld   a, b
    add  a, 8
    push bc
    ld   b, a              ; B = Y + 8
    ; C = X (bordo sinistro)
    call CheckWillyWallCollision
    pop  bc
    jr   c, .no_step_down  ; Se collide, non effettuare lo step down

    ld   a, b
    add  a, 8
    push bc
    ld   b, a              ; B = Y + 8
    ld   a, c
    add  a, 7
    ld   c, a              ; C = X + 7 (bordo destro)
    call CheckWillyWallCollision
    pop  bc
    jr   c, .no_step_down  ; Se collide, non effettuare lo step down

    ; Applica lo step down
```

```
    ld   a, b
    add  a, 8
    ld   b, a              ; B = Y + 8
    ld   (PlayerPos+1), a  ; Aggiorna Y

.no_step_down:

    ; Se on_ground == 1, controlliamo se siamo ancora su terreno solido
    ; Dobbiamo controllare i pixel direttamente sotto i piedi del player:
    ; (PlayerX, PlayerY + 16) e (PlayerX + 7, PlayerY + 16)
    ld  a, b
    add  a, 16
    ld  b, a              ; B = PlayerY + 16

    ; Controlla se la cella sotto il piede sinistro (PlayerX, PlayerY + 16) è solida
    call Phys_CheckSolid
    jr   c, .still_on_ground

    ; Controlla se la cella sotto il piede destro (PlayerX + 7, PlayerY + 16) è solida
    ld   a, c
    add  a, 7
    ld   c, a              ; C = PlayerX + 7
    call Phys_CheckSolid
    jr   c, .still_on_ground

    ; Se nessuna delle due è solida, iniziamo a cadere!
    xor  a
    ld   (on_ground), a
    ld   (velocity_y), a   ; Inizia la caduta con velocità 0
    jp   .in_air

.still_on_ground:
    ; Siamo ancora sulla piattaforma o sul pavimento
    ; Ripristina bc originale (PlayerPos)
    ld   bc, (PlayerPos)

    ; Gestisci il salto
    ld   a, (jump_pressed)
    or   a
    jp   z, .end

    xor  a
    ld   (Audio_jump_sound_note), a
    ld   (Audio_jump_sound_timer), a

    ld   a, (room_jump_strength)
    ld   (velocity_y), a
    xor  a
    ld   (on_ground), a
    ld   (jump_pressed), a
    ld   (jump_frame), a
    ld   a, b              ; B = PlayerY
    ld   (jump_start_y), a
    ld   a, (current_input_dir)
    ld   (jump_dir), a
    jp   .in_air

.in_air:
    ; Incrementa jump_frame per mantenere coerente la gravità frazionaria del salto
    ld   a, (jump_frame)
    inc  a
    ld   (jump_frame), a

    ; Leggi il preset della gravità
    ld   b, a                  ; Salva jump_frame in B
    ld   a, (room_gravity_preset)

    ; Controlla il preset
    or   a                     ; cp 0 (Gravità 0.5)
    jr   z, .grav_0_5
    dec  a                     ; cp 1 (Gravità 0.75 - Standard)
    jr   z, .grav_0_75
    dec  a                     ; cp 2 (Gravità 1.0)
    jr   z, .grav_1_0
    dec  a                     ; cp 3 (Gravità 0.25)
    jr   z, .grav_0_25
    dec  a                     ; cp 4 (Gravità 0.0)
    jr   z, .skip_gravity
    dec  a                     ; cp 5 (Gravità 0.875 - Intermedia tra 0.75 e 1.0)
    jr   z, .grav_0_85
```

```
    dec  a                    ; cp 6 (Gravità 0.625 - Intermedia tra 0.5 e 0.75)
    jr   z, .grav_0_65

    ; Default a 0.75 se valore sconosciuto
    jr   .grav_0_75

.grav_0_5:
    ld   a, b                 ; Ripristina jump_frame
    and  1
    jr   nz, .skip_gravity    ; Salta se bit 0 è impostato (media 0.5)
    jr   .apply_gravity

.grav_0_75:
    ld   a, b                 ; Ripristina jump_frame
    and  3
    cp   3
    jr   z, .skip_gravity     ; Salta se frame è 3 (media 0.75)
    jr   .apply_gravity

.grav_0_85:
    ld   a, b                 ; Ripristina jump_frame
    and  7
    cp   5
    jr   z, .skip_gravity     ; Salta se frame % 8 è 5 (media 0.875 -> altezza ~23.5px con Salto 6)
    jr   .apply_gravity

.grav_0_65:
    ld   a, b                 ; Ripristina jump_frame
    and  7
    cp   3
    jr   z, .skip_gravity
    cp   7
    jr   z, .skip_gravity     ; Salta se frame % 8 è 3 o 7 (media 0.75 alternata)
    jr   .apply_gravity

.grav_0_25:
    ld   a, b                 ; Ripristina jump_frame
    and  3
    cp   3
    jr   nz, .skip_gravity    ; Salta se frame non è 3 (media 0.25)
    jr   .apply_gravity

.grav_1_0:
    ; Non saltare mai la gravità
    jr   .apply_gravity

.apply_gravity:
    ld   a, (velocity_y)
    sub  1
    jr   .clamp_fall_speed

.skip_gravity:
    ld   a, (velocity_y)

.clamp_fall_speed:
    ; Controlla se positivo per evitare bug con cp signed
    bit 7, a
    jr  z, .store_vel

    cp  -MAX_FALL_SPEED   ; -12
    jr  nc, .store_vel
    ld  a, -MAX_FALL_SPEED

.store_vel:
    ld  (velocity_y), a

    ; Applica movimento verticale: PlayerY = PlayerY - velocity_y
    ld  bc, (PlayerPos)
    ld  d, a              ; D = velocity_y (signed)
    ld  a, b              ; A = PlayerY
    sub d                 ; A = PlayerY - velocity_y
    ld  b, a              ; B = new PlayerY

    ld  a, (velocity_y)
    bit 7, a              ; Se negativo, bit 7 = 1
    jr  nz, .falling_checks
    or  a                 ; Se zero
    jr  z, .falling_checks

    ; Se stiamo salendo, controlla la collisione della testa con il soffitto (muri Wall)
```

```
    ; In questo punto, bc contiene: C = PlayerX, B = new PlayerY
    push bc

    ; Controlla se la testa a sinistra (PlayerX, new_Y) colpisce un Wall
    call CheckWallCell
    jr   c, .ceiling_collision

    ; Controlla se la testa a destra (PlayerX + 7, new_Y) colpisce un Wall
    ld   a, c
    add  a, 7
    ld   c, a              ; C = PlayerX + 7
    call CheckWallCell
    jr   c, .ceiling_collision

    pop  bc
    jp   .no_collision_checks

.ceiling_collision:
    pop  bc                ; Ripristina bc originale (C = PlayerX, B = new PlayerY)

    ; Calcola la coordinata Y clampata appena sotto il soffitto.
    ; Il muro colpito è a riga r = new_Y / 8.
    ; Il limite Y consentito è riga r + 1, cioè Y = (r + 1) * 8.
    ld   a, b              ; A = new_Y
    srl  a
    srl  a
    srl  a                 ; A = r (0..15)
    inc  a                 ; r + 1
    add  a, a              ; *2
    add  a, a              ; *4
    add  a, a              ; *8 (Y clampato)

    ld   b, a              ; B = Y clampato

    ; Ferma la velocità verso l'alto (inizia la caduta)
    xor  a
    ld   (velocity_y), a
    jp   .save_pos

.falling_checks:
    ; --- CONTROLLO ATTERRAGGIO SU RAMPA ---
    push bc
    ; BC ha C = PlayerX, B = new_Y
    ld   a, (PlayerPos+1)   ; A = old_Y
    ld   d, a               ; D = old_Y

    call GetRampY           ; Calcola ramp_Y per PlayerX (in C)
    jp   nc, .no_ramp_landing

    ld   h, a               ; H = ramp_Y

    ; Se ramp_Y è negativo (bit 7 = 1, la rampa svetta oltre la cima dello schermo Y < 0),
    ; Willy (old_Y >= 0) è sicuramente SOTTO la rampa -> passa oltre ed evita agganci errati.
    bit  7, h
    jp   nz, .no_ramp_landing

    ; Controlla se il salto è iniziato da sotto la rampa
    ld   a, (jump_start_y)
    or   a
    jr   z, .jump_start_ok  ; Se 0, non siamo in salto (è una caduta normale), permetti atterraggio

    cp   h
    jr   z, .jump_start_ok
    jp   nc, .no_ramp_landing ; Se jump_start_y > ramp_Y (numerico Y > H), eravamo sotto la rampa -> passa oltre

.jump_start_ok:
    ld   a, d               ; A = old_Y
    cp   h
    jr   z, .check_new_y
    jp   nc, .no_ramp_landing ; Se old_Y > ramp_Y (era sotto la rampa), non atterra

.check_new_y:
    ld   a, b               ; A = new_Y (in B)
    cp   h
    jp   c, .no_ramp_landing ; Se new_Y < ramp_Y (non ha ancora toccato la rampa), non atterra

    ; Atterrato sulla rampa!
    pop  bc                 ; BC ha C = PlayerX, B = new_Y
    ld   b, h               ; B = ramp_Y
```

```
    ; Controlla se ramp_Y è negativo (bordo superiore dello schermo)
    bit  7, b
    jr   nz, .ramp_top_exit ; Se B è negativo (bit 7 = 1), Willy è in cima alla rampa al bordo superiore!

    ld   (PlayerPos), bc
    ld   a, 1
    ld   (on_ground), a
    IF ENABLE_RAMP_DEBUG_BARS != 0
    ld   (willy_on_ramp), a
    ENDIF
    xor  a
    ld   (velocity_y), a
    ld   (jump_start_y), a  ; Reset jump Y
    ld   (jump_dir), a
    ld   (jump_frame), a
    jp   .end

.ramp_top_exit:
    xor  a
    ld   (velocity_y), a
    ld   (jump_start_y), a
    ld   (jump_dir), a
    ld   (jump_frame), a
    jr   .save_pos          ; Esegui controllo transizione di stanza verso l'alto!

.no_ramp_landing:
    pop  bc
    ; Se Willy sta cadendo oltre il bordo inferiore (new_Y >= 113) e c'è una stanza sotto,
    ; salta il controllo del terreno a riga 15 ed esegui la transizione verso la nuova stanza!
    ld   a, b              ; B = new_Y
    cp   113
    jr   c, .check_floor_collision

    ld   hl, (room_ptr)
    ld   de, 238           ; Offset per stanza sotto
    add  hl, de
    ld   a, (hl)           ; A = ID stanza sotto
    cp   $FF
    jr   nz, .no_collision_checks

.check_floor_collision:
    ; --- CONTROLLO COLLISIONE CON IL TERRENO DURANTE LA CADUTA ---
    ; Controlliamo solo al piede effettivo (Y + 16) per evitare che Willy atterri su blocchi orizzontali di lato
    ld   a, b
    add  a, 16
    cp   128
    jr   c, .y_ok_fall
    ld   a, 127
.y_ok_fall:
    ld   b, a              ; B = Nuovo Y + 16 (clampato)

    ; Controlla se il piede sinistro tocca solido a Y + 16
    call Phys_CheckSolid
    jr   c, .check_landing

    ; Controlla se il piede destro tocca solido a Y + 16
    ld   a, c
    add  a, 7
    ld   c, a              ; C = PlayerX + 7
    call Phys_CheckSolid
    jr   c, .check_landing

    ; Nessun atterraggio
    jr   .no_collision_checks

.check_landing:
    ; Abbiamo toccato una cella solida durante la caduta!
    ld   a, b
    srl  a
    srl  a
    srl  a                 ; A = r (0..15), riga di layout della tile

    ld   e, a              ; E = r
    add  a, a              ; *2
    add  a, a              ; *4
    add  a, a              ; *8
    add  a, 3              ; Tolleranza di 3 pixel sulla superficie superiore della tile
    ld   d, a              ; D = r * 8 + 3 (limite per l'aggancio della superficie)

    ; Posizione precedente dei piedi: Vecchio Y + 16
```

```
    ld   a, (PlayerPos+1)  ; A = Vecchio Y
    add  a, 16             ; A = Vecchio Y + 16

    cp   d                 ; Confronta Vecchio Y + 16 con (r * 8 + 3)
    jr   z, .do_landing    ; Se == (entro i primi 3px dalla superficie), atterra
    jr   c, .do_landing    ; Se < (era sopra la superficie prima di cadere), atterra

    jr   .no_collision_checks ; Se era nella metà inferiore della tile (proveniva da sotto), NON atterrare! ; Se era sotto la su

.do_landing:
    ; Atterra sulla riga di layout E!
    ld   a, e
    add  a, a              ; *2
    add  a, a              ; *4
    add  a, a              ; *8
    sub  16             ; A = PlayerY di atterraggio

    ld   bc, (PlayerPos)   ; Ripristina C = PlayerX
    ld   b, a              ; B = Nuovo PlayerY

    ld   (PlayerPos), bc
    ld   a, 1
    ld   (on_ground), a
    xor  a
    ld   (velocity_y), a
    ld   (jump_start_y), a  ; Reset jump Y
    ld   (jump_dir), a
    ld   (jump_frame), a
    jp   .end

.no_collision_checks:
    ld   a, (velocity_y)
    ld   d, a                 ; Ripristina D = velocity_y (evita corruzione se proveniamo da .check_landing)
    ld   bc, (PlayerPos)
    ld   a, b
    sub  d                 ; A = Nuovo Y (calcolato in .store_vel)
    ld   b, a              ; B = Nuovo Y

.save_pos:
    ld   a, b
    bit  7, a              ; Se Y è negativo (sotto 0)
    jr   z, .check_max

    ; Willy sta andando sopra il bordo superiore (Y < 0)
    ; Controlla se c'è una stanza sopra
    ld   hl, (room_ptr)
    ld   de, 237           ; Offset per stanza sopra
    add  hl, de
    ld   a, (hl)           ; A = ID stanza sopra
    cp   $FF
    jr   z, .no_up_transition

    ; C'è una stanza sopra!
    ld   b, 96             ; Posiziona Willy a Y=96 (sopra il pavimento a riga 14 della nuova stanza)
    jp   TriggerRoomChange

.no_up_transition:
    ld   b, 0
    xor  a
    ld   (velocity_y), a
    ld   (jump_start_y), a
    ld   (jump_dir), a
    jr   .save_ok

.check_max:
    ld   a, b
    cp   113               ; Se Y >= 113, Willy sta cadendo sotto il bordo inferiore
    jr   c, .save_ok

    ; Willy sta andando sotto il bordo inferiore (Y >= 113)
    ; Controlla se c'è una stanza sotto
    ld   hl, (room_ptr)
    ld   de, 238           ; Offset per stanza sotto
    add  hl, de
    ld   a, (hl)           ; A = ID stanza sotto
    cp   $FF
    jr   z, .no_down_transition

    ; C'è una stanza sotto!
    ld   b, 4              ; Posiziona Willy a Y=4 (in cima alla nuova stanza, evita re-triggering di Y<=0)
```

```
    jp   TriggerRoomChange

.no_down_transition:
    ld   b, 112
    xor  a
    ld   (velocity_y), a
    ld   (jump_start_y), a
    ld   (jump_dir), a

.save_ok:
    ld   (PlayerPos), bc

.end:
    xor  a
    ld   (jump_pressed), a
    ret

; ---------------------------------------------------------------------
; Verifica la collisione tra player e nemico (bounding box a livello di pixel)
; Output: Carry flag impostato se collisione, altrimenti resettato
; ---------------------------------------------------------------------
CheckCollision:
    push hl
    push de
    push bc
    push ix

    ld   ix, ActiveEnemies
    ld   b, 8              ; 8 slot
.loop:
    ld   a, (ix+0)         ; Flags
    and  $80               ; Active?
    jr   z, .next

    ; 1. Controllo orizzontale: PlayerX - EnemyX + 7 in [0, 22]
    ld   a, (PlayerPos)    ; A = PlayerX
    sub  (ix+2)            ; A = PlayerX - EnemyX (Offset 2: Current X)
    add  a, 7              ; A = PlayerX - EnemyX + 7
    cp   23                ; Se >= 23, nessuna collisione con questo
    jr   nc, .next

    ; 2. Controllo verticale: PlayerY - EnemyY + 15 in [0, 30]
    ld   a, (PlayerPos+1)  ; A = PlayerY
    sub  (ix+3)            ; A = PlayerY - EnemyY (Offset 3: Current Y)
    add  a, 15             ; A = PlayerY - EnemyY + 15
    cp   31                ; Se >= 31, nessuna collisione con questo
    jr   nc, .next

    ; Collisione avvenuta!
    pop  ix
    pop  bc
    pop  de
    pop  hl
    scf                    ; Imposta carry flag
    ret

.next:
    ld   de, 11
    add  ix, de            ; Prossimo slot (11 byte)
    dec  b
    jr   nz, .loop

    pop  ix
    pop  bc
    pop  de
    pop  hl
    or   a                 ; Reset carry flag
    ret

; ---------------------------------------------------------------------
CheckConveyorOrRampCell:
    push hl
    push de
    push bc

    ; 1. Converti Y pixel in B a layout row: r = Y / 8
    ld   a, b
    cp   128
    jr   nc, .not_solid
    srl  a
```

```
    srl  a
    srl  a                  ; A = r (0..15)
    ld   d, a               ; D = r (layout row)

    ; 2. Converti X pixel in C a column: c = X / 8
    ld   a, c
    srl  a
    srl  a
    srl  a                  ; A = c (0..31)
    ld   e, a               ; E = c (column)

    ; --- CONTROLLO CONVEYOR ---
    ld   a, (conveyor_len)
    or   a
    jr   z, .not_solid      ; Se non c'è conveyor, fine

    ; Controlla se Row corrisponde
    ld   a, (conveyor_row)
    cp   d
    jr   nz, .not_solid     ; Row diversa

    ; Controlla se Col è compresa: c >= conveyor_col e c < conveyor_col + conveyor_len
    ld   a, (conveyor_col)
    cp   e
    jr   z, .is_conveyor    ; c == conveyor_col -> solido!
    jr   c, .c_greater      ; c > conveyor_col -> controlla limite superiore
    jr   .not_solid         ; c < conveyor_col
.c_greater:
    ld   b, a               ; B = conveyor_col
    ld   a, (conveyor_len)
    add  a, b               ; A = conveyor_col + conveyor_len
    cp   e
    jr   z, .not_solid      ; c == len (fuori)
    jr   c, .not_solid      ; c > len (fuori)
.is_conveyor:
    pop  bc
    pop  de
    pop  hl
    scf                     ; Set carry (solido)
    ret

.not_solid:
    pop  bc
    pop  de
    pop  hl
    or   a                  ; Clear carry
    ret

; ---------------------------------------------------------------------
; Applica la forza di trascinamento del conveyor se Willy vi è sopra
; ---------------------------------------------------------------------
Phys_ApplyConveyor:
    ld   a, (on_ground)
    or   a
    ret  z                  ; Se non è a terra, nessuna forza

    ld   a, (conveyor_len)
    or   a
    ret  z                  ; Se non c'è conveyor in questa stanza, fine

    ld   bc, (PlayerPos)    ; C = WillyX, B = WillyY

    ; Calcola screenRow dei piedi del player: RowPiedi = Y / 8
    ld   a, b
    add  a, 16
    srl  a
    srl  a
    srl  a                  ; A = (Y + 16) / 8

    ; Confronta con conveyor_row
    ld   d, a               ; D = RowPiedi
    ld   a, (conveyor_row)
    cp   d
    ret  nz                 ; Se non siamo sulla riga del conveyor, fine

    ; ColStart = WillyX / 8
    ld   a, c
    srl  a
    srl  a
    srl  a                  ; A = ColStart
```

```
    ld   e, a              ; E = ColStart

    ; ColEnd = (WillyX + 7) / 8
    ld   a, c
    add  a, 7
    srl  a
    srl  a
    srl  a                 ; A = ColEnd
    ld   d, a              ; D = ColEnd

    ; conveyor_col_end = conveyor_col + conveyor_len - 1
    ld   a, (conveyor_col)
    ld   hl, conveyor_len
    add  a, (hl)
    dec  a                 ; A = conveyor_col_end

    ; E <= conveyor_col_end (conveyor_col_end >= E)
    cp   e
    ret  c                 ; Se conveyor_col_end < E, fine

    ; D >= conveyor_col
    ld   a, (conveyor_col)
    ld   e, a
    ld   a, d
    cp   e
    ret  c                 ; Se D < conveyor_col, fine

    ; Willy è sul conveyor! Muovilo nella direzione del conveyor!
    ld   a, (conveyor_dir)
    or   a
    jr   z, .move_left

.move_right:
    ld   a, 2
    ld   (direction), a    ; Imposta direzione Willy a destra (2)
    ld   a, c
    add  a, 1              ; trascina di 1 pixel a destra per frame
    cp   PLAYER_MAX_X
    jr   c, .store_x
    jr   z, .store_x

    ; Controlla se c'è una stanza a destra
    ld   hl, (room_ptr)
    ld   de, 236           ; Offset per stanza a destra
    add  hl, de
    ld   a, (hl)           ; A = ID stanza a destra
    cp   $FF
    jr   z, .no_right_conveyor_transition

    ; C'è una stanza a destra!
    ld   c, PLAYER_MIN_X   ; Posiziona Willy a sinistra della nuova stanza
    jp   TriggerRoomChange ; Salta alla transizione

.no_right_conveyor_transition:
    ld   a, PLAYER_MAX_X
    jr   .store_x

.move_left:
    ld   a, 1
    ld   (direction), a    ; Imposta direzione Willy a sinistra (1)
    ld   a, c
    sub  1                 ; trascina di 1 pixel a sinistra per frame
    cp   PLAYER_MIN_X
    jr   nc, .store_x

    ; Controlla se c'è una stanza a sinistra
    ld   hl, (room_ptr)
    ld   de, 235           ; Offset per stanza a sinistra
    add  hl, de
    ld   a, (hl)           ; A = ID stanza a sinistra
    cp   $FF
    jr   z, .no_left_conveyor_transition

    ; C'è una stanza a sinistra!
    ld   c, PLAYER_MAX_X   ; Posiziona Willy a destra della nuova stanza
    jp   TriggerRoomChange ; Salta alla transizione (non ritorna qui)

.no_left_conveyor_transition:
    ld   a, PLAYER_MIN_X
```

```
.store_x:
    ld   c, a
    ld   (PlayerPos), bc   ; salva la nuova posizione
    ret

; ---------------------------------------------------------------------
; Verifica se Willy si trova sopra un blocco Conveyor attivo
; Output: Carry flag impostato se Willy è sul conveyor, altrimenti resettato
; ---------------------------------------------------------------------
CheckWillyOnConveyor:
    push bc
    push de
    push hl

    ld   a, (on_ground)
    or   a
    jr   z, .not_on_conveyor

    ld   a, (conveyor_len)
    or   a
    jr   z, .not_on_conveyor

    ld   bc, (PlayerPos)   ; C = WillyX, B = WillyY

    ; Calcola screenRow dei piedi del player: RowPiedi = (Y + 16) / 8
    ld   a, b
    add  a, 16
    srl  a
    srl  a
    srl  a                 ; A = (Y + 16) / 8

    ; Confronta con conveyor_row
    ld   d, a              ; D = RowPiedi
    ld   a, (conveyor_row)
    cp   d
    jr   nz, .not_on_conveyor

    ; ColStart = WillyX / 8
    ld   a, c
    srl  a
    srl  a
    srl  a                 ; A = ColStart
    ld   e, a              ; E = ColStart

    ; ColEnd = (WillyX + 7) / 8
    ld   a, c
    add  a, 7
    srl  a
    srl  a
    srl  a                 ; A = ColEnd
    ld   d, a              ; D = ColEnd

    ; conveyor_col_end = conveyor_col + conveyor_len - 1
    ld   a, (conveyor_col)
    ld   hl, conveyor_len
    add  a, (hl)
    dec  a                 ; A = conveyor_col_end

    ; E <= conveyor_col_end (conveyor_col_end >= E)
    cp   e
    jr   c, .not_on_conveyor

    ; D >= conveyor_col
    ld   a, (conveyor_col)
    ld   e, a
    ld   a, d
    cp   e
    jr   c, .not_on_conveyor

    ; Willy è sul conveyor! Imposta carry
    pop  hl
    pop  de
    pop  bc
    scf
    ret

.not_on_conveyor:
    pop  hl
    pop  de
    pop  bc
```

```
    or   a                ; Clear carry
    ret

; ---------------------------------------------------------------------
; Verifica se la cella (Col in E, Row in D) è sulla rampa, immediatamente sotto di essa,
; o sulla piattaforma adiacente in cima.
; Output: Carry flag impostato se è sulla rampa, altrimenti resettato
; ---------------------------------------------------------------------
IsCellOnRamp:
    push bc
    push hl

    ld   a, (ramp_len)
    or   a
    jr   z, .no

    ; 1. Verifica se Col (in E) è nel range orizzontale della rampa
    ld   a, (ramp_dir)
    or   a
    jr   z, .range_left
.range_right:
    ld   a, (ramp_col)
    ld   b, a             ; B = ramp_col
    ld   a, (ramp_len)
    add  a, b             ; A = ramp_col + ramp_len
    ld   c, a             ; C = limit_right (exclusive)

    ld   a, e             ; A = Col
    cp   b
    jr   c, .check_platform_case ; Col < ramp_col
    cp   c
    jr   nc, .check_platform_case ; Col >= limit_right

    ; Calcola B = Col - ramp_col
    ld   a, e
    sub  b
    ld   b, a             ; B = offset (0..ramp_len-1)

    ; SlopeRow = ramp_row - B
    ld   a, (ramp_row)
    sub  b
    ld   b, a             ; B = SlopeRow
    jr   .check_row_match

.range_left:
    ; Per rampa a sinistra, range è [ramp_col - ramp_len + 1, ramp_col]
    ld   hl, ramp_len
    ld   a, (ramp_col)
    sub  (hl)
    inc  a                ; A = ramp_col - ramp_len + 1
    ld   b, a             ; B = limit_left

    ld   a, (ramp_col)
    inc  a
    ld   c, a             ; C = limit_right (exclusive)

    ld   a, e             ; A = Col
    cp   b
    jr   c, .check_platform_case
    cp   c
    jr   nc, .check_platform_case

    ; Calcola B = ramp_col - Col
    ld   a, (ramp_col)
    sub  e
    ld   b, a             ; B = offset

    ; SlopeRow = ramp_row - B
    ld   a, (ramp_row)
    sub  b
    ld   b, a             ; B = SlopeRow

.check_row_match:
    ; D (Row) deve essere uguale a SlopeRow (in B) o SlopeRow + 1 (immediatamente sotto)
    ld   a, d             ; A = Row
    cp   b
    jr   z, .yes          ; Row == SlopeRow
    inc  b                ; B = SlopeRow + 1
    cp   b
    jr   z, .yes          ; Row == SlopeRow + 1
```

```
.check_platform_case:
    ; 2. Controlla se è sulla piattaforma adiacente in cima
    ld   a, (ramp_dir)
    or   a
    jr   z, .plat_left
.plat_right:
    ld   a, (ramp_col)
    ld   hl, ramp_len
    add  a, (hl)           ; A = ramp_col + ramp_len
    jr   .compare_plat
.plat_left:
    ld   a, (ramp_col)
    ld   hl, ramp_len
    sub  (hl)              ; A = ramp_col - ramp_len
.compare_plat:
    cp   e                 ; Compare with Col
    jr   nz, .no

    ld   a, (ramp_row)
    ld   hl, ramp_len
    sub  (hl)
    inc  a                 ; A = ramp_row - ramp_len + 1
    cp   d                 ; Compare with Row
    jr   z, .yes

.no:
    pop  hl
    pop  bc
    or   a
    ret

.yes:
    pop  hl
    pop  bc
    scf
    ret

; ---------------------------------------------------------------------
; Verifica se c'è un blocco Wall (tipo 2) alle coordinate pixel (X, Y)
; Input: C = X, B = Y
; Output: Carry flag impostato se è un Wall, altrimenti resettato
; ---------------------------------------------------------------------
CheckWallCell:
    push hl
    push de
    push bc

    ; 1. Controlla se Y (in B) è fuori dalla griglia di layout
    ld   a, b
    cp   128
    jr   nc, .not_wall

    ; 2. Calcola riga di layout: r = Y / 8
    srl  a
    srl  a
    srl  a                 ; A = r (0..15)

    ; 3. Calcola colonna di layout: c = X / 8 (in C)
    ld   e, a              ; E = r (Row)
    ld   a, c
    srl  a
    srl  a
    srl  a                 ; A = c (0..31)
    ld   c, a              ; C = c (Col)

    ; Se la cella è sulla rampa, non considerarla un muro!
    ld   d, e              ; D = Row
    ld   e, c              ; E = Col
    call IsCellOnRamp
    jr   c, .not_wall

    ld   a, d              ; A = Row
    ld   c, e              ; C = Col
    call GetCellFromUnpacked ; A = cell type

    ; Controlla se è Wall (2)
    cp   2
    jr   z, .is_wall
```

```
.not_wall:
    pop  bc
    pop  de
    pop  hl
    or   a                  ; Clear carry
    ret

.is_wall:
    pop  bc
    pop  de
    pop  hl
    scf                     ; Set carry
    ret


; ---------------------------------------------------------------------
; Controlla se Willy collide orizzontalmente con un Wall alle coordinate specificate
; Input: C = X pixel (da controllare), B = Y pixel (attuale)
; Output: Carry flag impostato se c'è collisione, altrimenti resettato
; ---------------------------------------------------------------------
CheckWillyWallCollision:
    push bc

    ; 1. Punto superiore: (X, Y)
    call CheckWallCell
    jr   c, .collision

    ; 2. Punto medio: (X, Y + 7)
    ld   a, b
    add  a, 7
    ld   b, a
    call CheckWallCell
    jr   c, .collision

    ; 3. Punto inferiore: (X, Y + 15)
    pop  bc
    push bc
    ld   a, b
    add  a, 15
    ld   b, a
    call CheckWallCell
    jr   c, .collision

    pop  bc
    or   a                  ; Reset carry (nessuna collisione)
    ret

.collision:
    pop  bc
    scf                     ; Set carry (collisione avvenuta)
    ret

; ---------------------------------------------------------------------
; Decomprime il layout della stanza (128 byte) in un buffer di 512 byte in RAM.
; Ciascuna cella occupa così 1 byte (valore 0..3), consentendo accessi O(1) velocissimi.
; ---------------------------------------------------------------------
UnpackRoomLayout:
    push hl
    push de
    push bc

    ld   hl, (room_ptr)     ; HL = inizio layout stanza (128 byte)
    ld   de, unpacked_layout ; DE = buffer RAM
    ld   b, 128             ; 128 byte
.byte_loop:
    ld   a, (hl)            ; Leggi byte compresso
    inc  hl

    ld   c, a               ; C = salvataggio

    ; Cella 0: bit 7-6
    rlca
    rlca
    and  3
    ld   (de), a
    inc  de

    ; Cella 1: bit 5-4
    ld   a, c
    rlca
```

```
    rlca
    rlca
    rlca
    and  3
    ld   (de), a
    inc  de

    ; Cella 2: bit 3-2
    ld   a, c
    rrca
    rrca
    and  3
    ld   (de), a
    inc  de

    ; Cella 3: bit 1-0
    ld   a, c
    and  3
    ld   (de), a
    inc  de

    djnz .byte_loop

    pop  bc
    pop  de
    pop  hl
    ret

; ---------------------------------------------------------------------
; Legge il tipo di cella dal buffer decompresso in RAM
; Input: A = Row (0..15), C = Col (0..31)
; Output: A = tipo di cella (0..3)
; Preserva tutti gli altri registri.
; ---------------------------------------------------------------------
GetCellFromUnpacked:
    push hl
    push de

    ld   l, a
    ld   h, 0
    add  hl, hl
    add  hl, hl
    add  hl, hl
    add  hl, hl
    add  hl, hl            ; HL = Row * 32
    ld   e, c
    ld   d, 0
    add  hl, de            ; HL = Row * 32 + Col
    ld   de, unpacked_layout
    add  hl, de
    ld   a, (hl)           ; A = cell type

    pop  de
    pop  hl
    ret

; ---------------------------------------------------------------------
; Phys_CheckRampOnRoomChange
; Controlla se nella nuova stanza (DE = room layout ptr) c'è una rampa
; allineata con la posizione d'ingresso di Willy (C = PlayerX, B = PlayerY).
; Output: Se allineato, modifica B = ramp_Y e C = PlayerX e imposta on_ground = 1.
; ---------------------------------------------------------------------
Phys_CheckRampOnRoomChange:
    push hl
    push de
    push bc

    ; Offsets per RampData: DE + 218
    ld   hl, 218
    add  hl, de            ; HL = RampData pointer (nella nuova stanza)

    ld   a, (hl)           ; A = Direction (0 = up-left, 1 = up-right)
    ld   (ramp_dir), a
    inc  hl
    ld   e, (hl)
    inc  hl
    ld   d, (hl)           ; DE = Location
    inc  hl
    ld   a, (hl)           ; A = Length
    ld   (ramp_len), a
```

```
    ; Controlla se DE (Location) è 0 o Length è 0
    ld   a, d
    or   e
    jp   z, .no_ramp_change
    ld   a, (ramp_len)
    or   a
    jp   z, .no_ramp_change

    ; Calcola Row e Col: Offset = DE - 24064
    ld   hl, -24064
    add  hl, de            ; HL = Offset
    ld   d, h
    ld   e, l

    ld   a, e
    and  31
    ld   (ramp_col), a     ; Col Start

    srl  h : rr l
    srl  h : rr l
    srl  h : rr l
    srl  h : rr l
    srl  h : rr l
    ld   a, l
    ld   (ramp_row), a     ; Row Start

    pop  bc                ; BC ha C = PlayerX, B = PlayerY
    push bc

    ; Calcola ramp_Y per PlayerX (in C) nella nuova stanza
    call GetRampY          ; Output: Carry SET se X è nel range rampa, A = ramp_Y
    jr   c, .on_ramp_x_ok

    ; PlayerX (in C) è fuori dal range orizzontale della rampa.
    ; Verifichiamo se C è vicino all'estremità della rampa (tolleranza 16 pixel).
    ld   a, (ramp_dir)
    or   a
    jr   z, .prox_dir_left

.prox_dir_right:
    ; Rampa sale a destra (up-right)
    ; X_left = ramp_col * 8
    ld   a, (ramp_col)
    add  a, a : add a, a : add a, a ; *8
    ld   e, a              ; E = X_left
    ld   a, c              ; A = PlayerX
    cp   e
    jr   nc, .check_prox_right_bound

    ; PlayerX < X_left
    ld   a, e
    sub  c                 ; A = X_left - PlayerX
    cp   17                ; Diff <= 16 px?
    jp   nc, .no_ramp_change
    pop  bc
    ld   c, e
    push bc
    jr   .retest_ramp_y

.check_prox_right_bound:
    ld   a, (ramp_len)
    add  a, a : add a, a : add a, a ; *8
    add  a, e              ; A = X_right_boundary
    ld   d, a              ; D = X_right_boundary
    ld   a, c              ; A = PlayerX
    sub  d                 ; A = PlayerX - X_right_boundary
    cp   17                ; Diff <= 16 px?
    jp   nc, .no_ramp_change
    pop  bc
    ld   c, d
    dec  c
    push bc
    jr   .retest_ramp_y

.prox_dir_left:
    ; Rampa sale a sinistra (up-left)
    ; X_left = (ramp_col - ramp_len + 1) * 8
    ld   a, (ramp_col)
    inc  a
```

```
    ld   hl, ramp_len
    sub  (hl)
    add  a, a : add a, a : add a, a ; *8
    ld   e, a              ; E = X_left

    ld   a, c              ; A = PlayerX
    cp   e
    jr   nc, .check_prox_left_bound

    ; PlayerX < X_left
    ld   a, e
    sub  c                 ; A = X_left - PlayerX
    cp   17                ; Diff <= 16 px?
    jp   nc, .no_ramp_change
    pop  bc
    ld   c, e
    push bc
    jr   .retest_ramp_y

.check_prox_left_bound:
    ld   a, (ramp_col)
    add  a, a : add a, a : add a, a ; *8
    add  a, 8              ; A = X_right_boundary
    ld   d, a              ; D = X_right_boundary
    ld   a, c              ; A = PlayerX
    sub  d                 ; A = PlayerX - X_right_boundary
    cp   17                ; Diff <= 16 px?
    jp   nc, .no_ramp_change
    pop  bc
    ld   c, d
    dec  c
    push bc
    jr   .retest_ramp_y

.retest_ramp_y:
    pop  bc
    push bc
    call GetRampY
    jp   nc, .no_ramp_change

.on_ramp_x_ok:
    ; C'è una rampa a PlayerX! Confronta PlayerY (in B) con ramp_Y (in A)
    ld   e, a              ; E = ramp_Y
    ld   a, b              ; A = PlayerY
    sub  e                 ; A = PlayerY - ramp_Y
    bit  7, a
    jr   z, .abs_change
    neg
.abs_change:
    cp   16                ; Tolleranza 16 pixel per cambi stanza verticali
    jp   nc, .no_ramp_change

    ; Controllo bordo inferiore (ramp_Y >= 104)
    ld   a, e              ; A = ramp_Y
    cp   104
    jr   nc, .adj_bottom

    ; Controllo bordo superiore (ramp_Y <= 4 o negativo)
    bit  7, a
    jr   nz, .adj_top
    cp   5
    jr   c, .adj_top
    jr   .apply_ramp_align

.adj_bottom:
    ; ramp_Y >= 104 (bordo inferiore). Sposta X lungo la pendenza verso l'alto.
    ld   a, (ramp_dir)
    or   a
    jr   z, .adj_bot_left
.adj_bot_right:
    pop  bc
    ld   a, c
    add  a, 8
    ld   c, a
    push bc
    jr   .recalc_ramp_y
.adj_bot_left:
    pop  bc
    ld   a, c
    sub  8
```

```
    ld   c, a
    push bc
    jr   .recalc_ramp_y

.adj_top:
    ; ramp_Y <= 4 (bordo superiore). Sposta X lungo la pendenza verso il basso.
    ld   a, (ramp_dir)
    or   a
    jr   z, .adj_top_left
.adj_top_right:
    pop  bc
    ld   a, c
    sub  8
    ld   c, a
    push bc
    jr   .recalc_ramp_y
.adj_top_left:
    pop  bc
    ld   a, c
    add  a, 8
    ld   c, a
    push bc

.recalc_ramp_y:
    call GetRampY
    jp   nc, .no_ramp_change
    ld   e, a              ; E = nuovo ramp_Y safe

    ; Controlla se ancora ai bordi
    cp   104
    jr   nc, .adj_bottom
    bit  7, a
    jr   nz, .adj_top
    cp   5
    jr   c, .adj_top

.apply_ramp_align:
    pop  bc
    ld   b, e              ; B = ramp_Y
    push bc

    ld   a, 1
    ld   (on_ground), a    ; Imposta Willy a terra sulla rampa
    xor  a
    ld   (ramp_entry_guard), a

.no_ramp_change:
    pop  bc
    pop  de
    pop  hl
    ret

; ---------------------------------------------------------------------
; GetRampY
; Calcola la coordinata Y teorica di Willy se si trova sulla rampa
; Input: C = PlayerX
; Output: Carry flag impostato se PlayerX è nel range della rampa,
;         in tal caso A = PlayerY (Y teorico del player sulla rampa)
; ---------------------------------------------------------------------
GetRampY:
    ld   a, (ramp_len)
    or   a
    ret  z                 ; Se non c'è rampa (len=0), return carry clear

    ld   a, (ramp_dir)
    or   a
    jr   z, .dir_left

.dir_right:
    ; Rampa sale a destra (up-right)
    ld   a, (ramp_col)
    add  a, a              ; *2
    add  a, a              ; *4
    add  a, a              ; *8 (X start in pixel)
    ld   e, a              ; E = X_left

    ld   a, (ramp_len)
    add  a, a              ; *2
    add  a, a              ; *4
    add  a, a              ; *8
```

```
    add  a, e             ; A = (ramp_col + ramp_len) * 8 = X_right_boundary
    ld   h, a             ; H = X_right_boundary
    jr   c, .no_right_bound_check_right

    ld   a, c             ; A = PlayerX
    cp   e
    jp   c, .out_of_ramp  ; Se PlayerX < X_left, fuori

    cp   h
    jp   c, .on_ramp_slope ; Se PlayerX < X_right_boundary, pendenza rampa

    ; Controlla se è sulla cella della piattaforma adiacente in cima: [X_right_boundary, X_right_boundary + 7]
    sub  h                ; A = PlayerX - X_right_boundary
    cp   8
    jp   nc, .out_of_ramp  ; Se PlayerX >= X_right_boundary + 8, fuori
    jr   .on_platform_right

.no_right_bound_check_right:
    ld   a, c             ; A = PlayerX
    cp   e
    jp   c, .out_of_ramp  ; Se PlayerX < X_left, fuori
    jp   .on_ramp_slope   ; Se >= X_left, è sicuramente sulla pendenza rampa (non può superare 256)

.on_platform_right:
    ; Piattaforma adiacente! Restituisci PlayerY = (ramp_row - ramp_len + 1) * 8 - 16
    ld   a, (ramp_row)
    ld   hl, ramp_len
    sub  (hl)
    inc  a                ; A = Row
    add  a, a             ; *2
    add  a, a             ; *4
    add  a, a             ; *8
    sub  16               ; A = Row * 8 - 16
    jp   .return_ramp_y

.on_ramp_slope:
    ld   a, c             ; A = PlayerX
    sub  e                ; A = dx (PlayerX - X_left)
    ld   d, a             ; D = dx

    ld   a, (ramp_row)
    add  a, a             ; *2
    add  a, a             ; *4
    add  a, a             ; *8
    sub  16               ; A = ramp_row * 8 - 16 (base Y)
    sub  d                ; A = (ramp_row * 8 - 16) - dx
    ld   d, a             ; D = calculated Y

    ; H = platform_Y + 16 (confronto sicuro con offset +16)
    ld   a, (ramp_row)
    ld   hl, ramp_len
    sub  (hl)
    inc  a                ; A = Row
    add  a, a             ; *2
    add  a, a             ; *4
    add  a, a             ; *8
    ld   h, a             ; H = platform_Y + 16

    ld   a, d             ; A = calculated Y
    add  a, 16            ; A = calculated Y + 16
    cp   h                ; Compare (calculated Y + 16) con (platform_Y + 16)
    jr   nc, .no_clamp    ; Se calculated Y >= platform_Y, non limitare

    ; Altrimenti, limita a platform_Y
    ld   a, h
    sub  16               ; A = platform_Y
    jp   .return_ramp_y

.no_clamp:
    ld   a, d             ; Ripristina A = calculated Y
    jp   .return_ramp_y

.dir_left:
    ; Rampa sale a sinistra (up-left)
    ld   a, (ramp_col)
    inc  a                ; A = ramp_col + 1
    ld   hl, ramp_len
    cp   (hl)             ; Compare (ramp_col + 1) con ramp_len
    jp   c, .out_of_ramp  ; Se ramp_len > ramp_col + 1, evita underflow!
    sub  (hl)             ; A = ramp_col - ramp_len + 1
```

```
    add  a, a              ; *2
    add  a, a              ; *4
    add  a, a              ; *8
    ld   e, a              ; E = X_left

    ld   a, (ramp_col)
    add  a, a              ; *2
    add  a, a              ; *4
    add  a, a              ; *8
    add  a, 8              ; A = (ramp_col + 1) * 8 (X_right_boundary)
    ld   h, a              ; H = X_right_boundary
    jr   c, .no_right_bound_check_left

    ld   a, c              ; A = PlayerX
    cp   h
    jp   nc, .out_of_ramp  ; Se PlayerX >= X_right_boundary, fuori

.no_right_bound_check_left:
    ld   a, c              ; A = PlayerX
    cp   e
    jr   nc, .on_ramp_slope_left ; Se PlayerX >= X_left, pendenza rampa

    ; Controlla se è sulla cella della piattaforma adiacente in cima a sinistra: [X_left - 8, X_left - 1]
    ld   a, e              ; A = X_left
    sub  c                 ; A = X_left - PlayerX
    cp   9
    jp   nc, .out_of_ramp  ; Se PlayerX <= X_left - 9, fuori

    ; Piattaforma adiacente a sinistra! Restituisci PlayerY = (ramp_row - ramp_len + 1) * 8 - 16
    ld   a, (ramp_row)
    ld   hl, ramp_len
    sub  (hl)
    inc  a                 ; A = Row
    add  a, a              ; *2
    add  a, a              ; *4
    add  a, a              ; *8
    sub  16                ; A = Row * 8 - 16
    jp   .return_ramp_y

.on_ramp_slope_left:
    ld   a, (ramp_col)
    add  a, a              ; *2
    add  a, a              ; *4
    add  a, a              ; *8
    ld   d, a              ; D = X_start (ramp_col * 8)

    ld   a, d              ; A = X_start
    sub  c                 ; A = dx (X_start - PlayerX)
    ld   e, a              ; E = dx

    ld   a, (ramp_row)
    add  a, a              ; *2
    add  a, a              ; *4
    add  a, a              ; *8
    sub  16                ; A = ramp_row * 8 - 16 (base Y)
    sub  e                 ; A = (ramp_row * 8 - 16) - dx
    ld   d, a              ; D = calculated Y

    ; H = platform_Y + 16 (confronto sicuro con offset +16)
    ld   a, (ramp_row)
    ld   hl, ramp_len
    sub  (hl)
    inc  a                 ; A = Row
    add  a, a              ; *2
    add  a, a              ; *4
    add  a, a              ; *8
    ld   h, a              ; H = platform_Y + 16

    ld   a, d              ; A = calculated Y
    add  a, 16             ; A = calculated Y + 16
    cp   h                 ; Compare (calculated Y + 16) con (platform_Y + 16)
    jr   nc, .no_clamp_left ; Se calculated Y >= platform_Y, non limitare

    ; Altrimenti, limita a platform_Y
    ld   a, h
    sub  16                ; A = platform_Y
    jp   .return_ramp_y

.no_clamp_left:
    ld   a, d
```

```
    jp   .return_ramp_y

.return_ramp_y:
    ; A contiene la coordinata ramp_Y calcolata per PlayerX (in C).
    ; Verifichiamo se la guardia ingresso orizzontale è ancora attiva.
    push bc
    ld   b, a                  ; B = ramp_Y
    ld   a, (ramp_entry_guard)
    or   a
    jr   z, .guard_clear_exit  ; Se la guardia è 0, restituisci normalmente A = ramp_Y, Carry SET

    ; La guardia è attiva!
    ld   a, (ramp_entry_guard_dir)
    cp   1
    jr   z, .check_x_left

.check_x_right:
    ld   a, c                  ; A = PlayerX
    ld   hl, ramp_entry_guard_x
    cp   (hl)                  ; PlayerX - ramp_entry_guard_x
    jr   c, .deactivate_guard  ; Se PlayerX < ramp_entry_guard_x, Willy è uscito dalla zona di guardia -> disattiva
    jr   .check_alignment

.check_x_left:
    ld   a, c                  ; A = PlayerX
    ld   hl, ramp_entry_guard_x
    cp   (hl)                  ; PlayerX - ramp_entry_guard_x
    jr   nc, .deactivate_guard ; Se PlayerX >= ramp_entry_guard_x, Willy è uscito dalla zona di guardia -> disattiva

.check_alignment:
    ; Willy si trova nella zona d'ingresso (es. X < 16 o X >= 232).
    ; Verifichiamo se Willy è GIÀ ALLINEATO alla rampa (stava camminando/atterrando su di essa).
    ; Confrontiamo (PlayerPos+1) (PlayerY) con B (ramp_Y).
    ld   a, (PlayerPos+1)      ; A = PlayerY
    sub  b                     ; A = PlayerY - ramp_Y
    bit  7, a
    jr   z, .diff_positive
    neg
.diff_positive:
    cp   12                    ; Se |PlayerY - ramp_Y| < 12, Willy è allineato alla rampa!
    jr   c, .deactivate_guard  ; È sulla rampa che prosegue -> disattiva guardia e permetti di proseguire!

    ; Willy NON è allineato (es. sta camminando sul pavimento sotto la rampa).
    pop  bc
    or   a                     ; Reset carry (ignora rampa!)
    ret

.deactivate_guard:
    xor  a
    ld   (ramp_entry_guard), a

.guard_clear_exit:
    ld   a, b                  ; A = ramp_Y
    pop  bc
    scf                        ; Set carry (rampa valida!)
    ret

.out_of_ramp:
    or   a                ; Clear carry
    ret

; ----------------------------------------------------------------------
; Carica le impostazioni di salto e gravità specifiche per la stanza corrente
; Legge il byte di configurazione ad offset OFFSET_PHYSICS_SETTINGS (239).
; Se è 0, applica le impostazioni di default (Jump = 6, Gravity = 1).
; Altrimenti, decodifica il formato %GGGGJJJJ (Jump = JJJJ, Gravity Preset = GGGG).
; ----------------------------------------------------------------------
Phys_LoadRoomSettings:
    push hl
    push de
    push af

    ld   hl, (room_ptr)
    ld   de, OFFSET_PHYSICS_SETTINGS
    add  hl, de
    ld   a, (hl)
    or   a
    jr   nz, .custom_settings

    ld   a, DEFAULT_JUMP_STRENGTH
```

```
    ld   (room_jump_strength), a
    ld   a, DEFAULT_GRAVITY_PRESET
    ld   (room_gravity_preset), a
    jr   .done

.custom_settings:
    push af
    and  $0f                  ; Maschera i 4 bit inferiori (JJJJ)
    ld   (room_jump_strength), a
    pop  af
    srl  a
    srl  a
    srl  a
    srl  a                  ; Sposta i 4 bit superiori in quelli inferiori (GGGG)
    ld   (room_gravity_preset), a

.done:
    pop  af
    pop  de
    pop  hl
    ret

; ---------------------------------------------------------------------
; Verifica se c'è una cella Nasty (tipo 3) alle coordinate pixel (X, Y)
; Input: C = X, B = Y
; Output: Carry flag impostato se è Nasty, altrimenti resettato
; ---------------------------------------------------------------------
CheckNastyCell:
    push hl
    push de
    push bc

    ; 1. Controlla se Y (in B) è fuori dalla griglia di layout
    ld   a, b
    cp   128
    jr   nc, .not_nasty

    ; 2. Calcola riga di layout: r = Y / 8
    srl  a
    srl  a
    srl  a                  ; A = r (0..15)

    ; 3. Calcola colonna di layout: c = X / 8 (in C)
    ld   e, a               ; E = r
    ld   a, c
    srl  a
    srl  a
    srl  a                  ; A = c (0..31)
    ld   c, a               ; C = c
    ld   a, e               ; A = r

    call GetCellFromUnpacked ; A = cell type

    ; Controlla se è Nasty (3)
    cp   3
    jr   z, .is_nasty

.not_nasty:
    pop  bc
    pop  de
    pop  hl
    or   a                  ; Clear carry
    ret

.is_nasty:
    pop  bc
    pop  de
    pop  hl
    scf                     ; Set carry
    ret

; ---------------------------------------------------------------------
; Controlla la collisione di Willy con le celle Nasty (tipo 3)
; Controlla 6 punti strategici della bounding box di Willy (16x8 pixel)
; Output: Carry flag impostato se c'è collisione, altrimenti resettato
; ---------------------------------------------------------------------
Phys_CheckNastyCollision:
    push bc

    ld   bc, (PlayerPos)   ; B = Y, C = X
```

```
    ; Punto 1: Top-Left (X, Y)
    call CheckNastyCell
    jr   c, .collision

    ; Punto 2: Top-Right (X + 7, Y)
    ld   a, c
    add  a, 7
    ld   c, a
    call CheckNastyCell
    jr   c, .collision

    ; Punto 3: Mid-Left (X, Y + 7)
    ld   bc, (PlayerPos)
    ld   a, b
    add  a, 7
    ld   b, a
    call CheckNastyCell
    jr   c, .collision

    ; Punto 4: Mid-Right (X + 7, Y + 7)
    ld   a, c
    add  a, 7
    ld   c, a
    call CheckNastyCell
    jr   c, .collision

    ; Punto 5: Bottom-Left (X, Y + 15)
    ld   bc, (PlayerPos)
    ld   a, b
    add  a, 15
    ld   b, a
    call CheckNastyCell
    jr   c, .collision

    ; Punto 6: Bottom-Right (X + 7, Y + 15)
    ld   a, c
    add  a, 7
    ld   c, a
    call CheckNastyCell
    jr   c, .collision

    pop  bc
    or   a                ; Clear carry
    ret

.collision:
    pop  bc
    scf                   ; Set carry
    ret
```